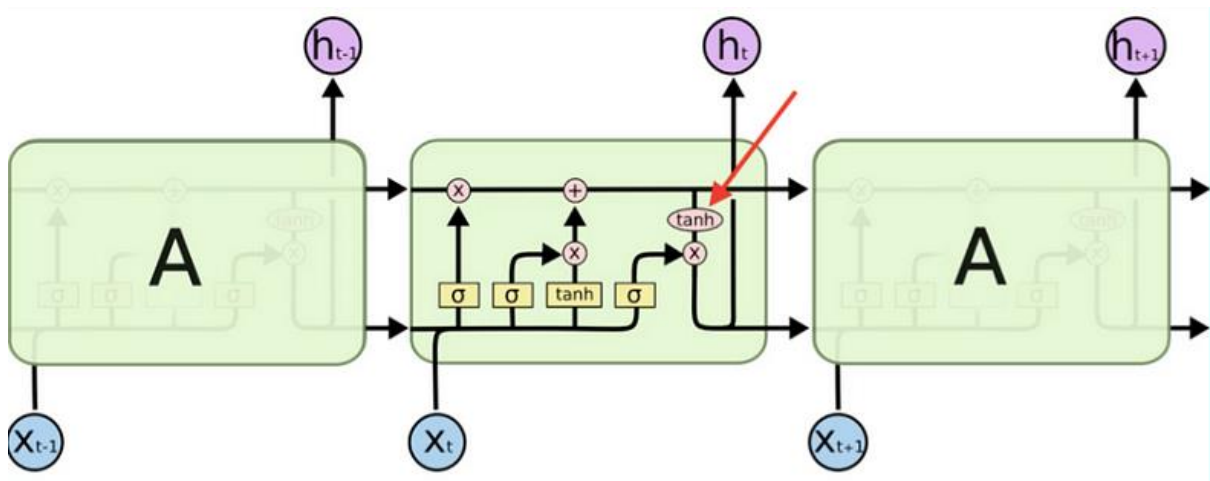


LSTM Notes



In the world of artificial intelligence and machine learning, Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture known for its ability to effectively capture long-term dependencies in sequential data. LSTM achieves this by introducing specialized units called "gates" within its architecture. These gates regulate the flow of information through the network, allowing it to retain or discard information as needed. Let's explore each gate in detail:

1. Forget Gate:

- Purpose: The forget gate determines which information from the previous time step should be discarded from the cell state (long-term memory).

- Operation: It takes input from the previous hidden state (h_{t-1}) and the current input (x_t), passes them through a sigmoid activation function, and produces an output f_t (the forget gate activation vector) between 0 and 1 for each element. This output is then multiplied element-wise with the previous cell state (c_{t-1}), where a value close to 0 indicates "forget" and close to 1 indicates "remember".

2. Input Gate:

- Purpose: The input gate decides which new information should be stored in the cell state.

- Operation: It consists of two sub-steps:

- Gathering information: It takes input from the previous hidden state (h_{t-1}) and the current input (x_t), passes them through a sigmoid activation function to generate a candidate update vector (i_t).

- Filtering information: It takes input from the previous hidden state (h_{t-1}) and the current input (x_t), passes them through a tanh activation function to generate a candidate cell state (c_t). The two outputs are multiplied element-wise, and the resulting vector is used to update the cell state (c_t).

3. Output Gate:

- Purpose: The output gate determines which information from the current cell state should be exposed to the next hidden state.

- Operation: It takes input from the previous hidden state (h_{t-1}) and the current input (x_t), passes them through a sigmoid activation function to generate an output gate activation vector (o_t). The current cell state (c_t) is then passed through a tanh activation function to squash the values between -1 and 1. The output gate activation vector is multiplied element-wise with the squashed cell state to produce the next hidden state (h_t).

These gates, working in harmony, allow LSTM to effectively manage information flow, capture long-term dependencies, and mitigate the vanishing gradient problem commonly encountered in traditional RNNs. By selectively remembering, forgetting, and updating information, LSTM can excel in tasks involving sequential data, such as natural language processing, time series prediction, and speech recognition.

Storytelling

Once upon a time, in a bustling school nestled in a colourful town, there was a remarkable student named LSTM. LSTM wasn't like any other student – it had a special gift. It could remember important lessons and knowledge while letting go of what wasn't needed anymore.

Within LSTM's schoolbag, there were three magical gates: Forget, Input, and Output. These gates were like guardians of LSTM's memories, each with its own important task.

Let's start with the Forget Gate. It was a sturdy gate, gleaming with gold. Its job was to decide which memories to bid farewell to. Guarding this gate were two friends: h_t and c_t .

h_t was quick and clever, always on the lookout for memories to let go of. c_t was wise and knew which memories LSTM didn't need anymore.

Every day, as the sun rose, the Forget Gate would open. h_t and c_t would carefully examine the memories stored in LSTM's mind. If a memory wasn't important anymore – perhaps an old equation or a forgotten historical date – ft , the keeper of forgetfulness, would make it vanish.

But amidst the forgetting, there was always room for new knowledge. The Input Gate eagerly awaited fresh lessons and stories. x_t , a cheerful messenger, would bring news from the classrooms. If a lesson was exciting and worth remembering, c_t , the guardian of inclusion, would ensure it was kept safe in LSTM's memory.

Yet, memories weren't just for storing; they were for using too. The Output Gate decided which memories to utilize. o_t , the memory conductor, carefully selected the best ones to create something amazing – perhaps a brilliant answer during a test or a creative solution to a problem.

So, within LSTM's mind, the gates worked tirelessly to keep important lessons and let go of the rest. It made LSTM a very clever and happy student, ready to tackle any challenge that came its way.

Let's consider a simplified mathematical example to illustrate the workings of LSTM gates:

Imagine LSTM's memory as a list of numbers representing daily temperatures over a week: [20, 22, 25, 28, 26, 24, 23]. Each number represents a memory LSTM needs to manage.

1. Forget Gate: LSTM needs to decide which temperatures to forget from the previous week. Let's say the Forget Gate's output, f_t , is calculated using a sigmoid function. For simplicity, let's assume $f_t = [0.8, 0.7, 0.5, 0.3, 0.6, 0.7, 0.9]$. This means LSTM should remember 80% of the temperature from the first day, 70% from the second day, and so on.

2. Input Gate: Now, LSTM receives new temperature data for the current week: [22, 24, 27, 30, 28, 26, 25]. x_t , the input, brings this new data. The Input Gate decides which temperatures to include in LSTM's memory. Let's say the Input Gate's output, i_t , is calculated using another sigmoid function. For simplicity, let's assume $i_t = [0.6, 0.8, 0.9, 0.7, 0.8, 0.6, 0.5]$. This means LSTM should include 60% of the temperature from the first day of the new week, 80% from the second day, and so on.

3. Output Gate: Finally, LSTM needs to decide which memories to use to predict the temperature for the upcoming day. Let's say LSTM decides to use the memories from the previous week, i.e., [20, 22, 25, 28, 26, 24, 23], and weights them with O_t , the Output Gate's output, calculated using a sigmoid function. For simplicity, let's assume $O_t = [0.7, 0.6, 0.8, 0.9, 0.7, 0.6, 0.5]$. This means LSTM will give more weight to memories from days with higher O_t values when predicting the temperature for the next day.

The concepts of nodes, activation functions, and pointwise operations in the context of LSTM:

1. Nodes:

- In the context of neural networks, a node (or neuron) is a fundamental computational unit that processes and transmits information. Each node receives inputs, performs computations on them, and produces an output.

- In an LSTM network, nodes are organized into layers. There are typically three types of layers: input layer, hidden layers, and output layer. Each layer consists of multiple nodes, and connections between nodes carry information flow.

2. Activation Functions:

- Activation functions are mathematical functions applied to the output of nodes in neural networks. They introduce non-linearity to the network, enabling it to learn complex patterns and relationships in data.

- In an LSTM network, several activation functions are used:

- Sigmoid Activation Function: It squashes the input values between 0 and 1, making it suitable for gating mechanisms in LSTM. It is often used in forget gates, input gates, and output gates.

- Hyperbolic Tangent (tanh) Activation Function: It squashes the input values between -1 and 1, allowing the network to capture both positive and negative

information. It is commonly used to regulate the candidate cell state in the input gate and to produce the output of the cell state.

- Identity Activation Function: In some cases, an identity activation function ($f(x) = x$) is used, especially in the output layer when the desired output range is not restricted.

3. Pointwise Operations:

- Pointwise operations refer to operations performed element-wise on the vectors or matrices involved in the computation. In the context of LSTM:

- Pointwise addition: Involves adding corresponding elements of two vectors or matrices element-wise. For example, in LSTM, the forget gate output (ft) and the previous cell state ($ct-1$) are multiplied element-wise, representing which information to forget.

- Pointwise multiplication: Involves multiplying corresponding elements of two vectors or matrices element-wise. For example, in LSTM, the input gate output (it) and the candidate cell state ($c \square t$) are multiplied element-wise, representing the information to be added to the cell state.

- Pointwise activation: Involves applying activation functions to each element of a vector or matrix independently. For example, in LSTM, the sigmoid activation function is applied element-wise to the output of the forget gate, input gate, and output gate to obtain values between 0 and 1.

These concepts are fundamental to understanding the inner workings of LSTM networks and how they process information through nodes, apply activation functions to introduce non-linearity, and perform pointwise operations to regulate information flow.

Storytelling

Once upon a time, in a bustling village in India, there was a clever young student named LSTM. LSTM was on a quest to learn about the magical world of neural networks, where nodes, activation functions, and pointwise operations held the key to unlocking powerful secrets.

1. ****Nodes - The Messengers****:

Imagine LSTM's village as a bustling marketplace, where messages needed to be passed from one end to another. Each node in a neural network was like a messenger in the village. They carried information from one place to another, making sure everyone knew what was happening. Just like how villagers interacted with each other, nodes in the network communicated with each other, sharing important information.

2. ****Activation Functions - The Magic Spells****:

In LSTM's village, there were wise elders who knew special spells that could transform things magically. These spells were like activation functions in neural networks. One spell, called the "Squash Spell," could squeeze big numbers into small ones between 0 and 1, just like the sigmoid activation function. Another spell, known as the "Balancing

Spell," could balance things between -1 and 1, just like the tanh activation function. Whenever the villagers needed to transform numbers, they would call upon these wise elders to cast their magical spells.

3. ****Pointwise Operations - The Mixing Pot****:

In the heart of the village, there was a big mixing pot where villagers combined different ingredients to make delicious dishes. This mixing pot represented pointwise operations in neural networks. Villagers would add spices, vegetables, and meats, then mix them together to create a flavourful meal. Similarly, in neural networks, pointwise operations involved mixing different numbers element by element. Sometimes they would add, sometimes they would multiply, and sometimes they would apply magical spells (activation functions) to each ingredient separately, just like the villagers did in their mixing pot.

So, in LSTM's journey to understand neural networks, nodes acted as messengers, activation functions were like magical spells, and pointwise operations resembled the mixing pot in the village. With this newfound knowledge, LSTM could navigate the complexities of neural networks and uncover their hidden mysteries.

The mathematical equations:

Nodes:

- Weighted sum calculation:

$$z = \sum (w_i * x_i) + b$$

- Output calculation:

$$a = f(z)$$

Activation Functions:

- Sigmoid Activation Function:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

- Hyperbolic Tangent (tanh) Activation Function:

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

- Identity Activation Function (for simplicity):

$$f(x) = x$$

Pointwise Operations:

- Pointwise Addition:

$$\text{result}[i] = \text{vector1}[i] + \text{vector2}[i]$$

- Pointwise Multiplication:

$$\text{result}[i] = \text{vector1}[i] * \text{vector2}[i]$$

- Pointwise Activation:

result[i] = f(vector[i])

Long Short-Term Memory (LSTM) Algorithm:

1. **Initialization:**

- Initialize weights and biases for all gates and cells.
- Set initial hidden state h_0 and initial cell state c_0 to zeros.

2. **For each time step t in the input sequence:**

a. Compute the forget gate activation:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

b. Compute the input gate activation:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

c. Compute the candidate cell state:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

d. Update the cell state:

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$$

e. Compute the output gate activation:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

f. Compute the hidden state:

$$h_t = o_t \cdot \tanh(c_t)$$

3. **Output:**

- Return the sequence of hidden states $[h_1, h_2, \dots, h_T]$ as the output.

Note:

- W_f, W_i, W_c, W_o are weight matrices for the forget gate, input gate, candidate cell state, and output gate respectively.
- b_f, b_i, b_c, b_o are bias vectors for the forget gate, input gate, candidate cell state, and output gate respectively.
- σ represents the sigmoid activation function.
- $[h_{t-1}, x_t]$ denotes the concatenation of the previous hidden state h_{t-1} and the current input x_t .

Numerical Example: LSTM Cell Computation

Given Data:

- Input sequence (x_t):

$$- x_1 = 0.5$$

$$- x_2 = 0.8$$

$$- x_3 = 0.3$$

Parameters:

- Forget gate weights (W_f): 0.4

- Forget gate bias (b_f): 0.2

- Input gate weights (W_i): 0.6

- Input gate bias (b_i): 0.3

- Candidate cell state weights (W_c): -0.7

- Candidate cell state bias (b_c): 0.1

- Output gate weights (W_o): 0.5

- Output gate bias (b_o): -0.1

****Solution:****

1. For time step 1 ($t = 1$):

a. Compute the forget gate activation:

$$f_1 = \sigma(0.2 + 0.2)$$

$$\approx \sigma(0.4)$$

$$\approx 0.6$$

b. Compute the input gate activation:

$$i_1 = \sigma(0.3 + 0.3)$$

$$\approx \sigma(0.6)$$

$$\approx 0.65$$

c. Compute the candidate cell state:

$$c_1 = \tanh(-0.25)$$

$$\approx -0.24$$

d. Update the cell state:

$$c_1 = 0.6 * 0 + 0.65 * (-0.24)$$

$$\approx -0.156$$

e. Compute the output gate activation:

$$o_1 = \sigma(0.25 - 0.1)$$

$$\approx \sigma(0.15)$$

$$\approx 0.54$$

f. Compute the hidden state:

$$h_1 = 0.54 * (-0.155)$$

$$\approx -0.0837$$

Similar computations can be performed for time steps 2 and 3.